

Tema 2 – Procesos de Desarrollo de Software

Este documento no sustituye a los apuntes del tema ni a los ejercicios como tests, o actividades H5P. Es únicamente un documento útil para el repaso.

Objetivos de aprendizaje

- 2.1.** Enumerar las actividades habituales de un proceso de desarrollo y explicar por qué el coste de un error crece cuanto más tarde se detecta.
- 2.2.** Distinguir cascada y modelo en V, y justificar en qué tipo de proyecto encaja cada uno.
- 2.3.** Diferenciar iteración de incremento, y explicar los tres pilares del UP (dirigido por casos de uso, centrado en arquitectura, iterativo e incremental) y su organización en Inicio/Elaboración/Construcción/Transición.
- 2.4.** Explicar los valores del Manifiesto Ágil y comparar Scrum y XP en roles, artefactos y prácticas.
- 2.5.** Justificar para qué sirve la gestión de configuración y describir el ciclo commit–branch–merge.
- 2.6.** Diferenciar CI, entrega continua y despliegue continuo, y situar CI/CD dentro de DevOps.
- 2.7.** Distinguir la IA como parte del producto frente a la IA como herramienta del proceso, y argumentar por qué la responsabilidad final es siempre humana.

Esenciales por bloque

2.1 · Proceso de desarrollo

- Actividades comunes: requisitos, análisis, diseño, implementación, pruebas, integración, mantenimiento/evolución (la frontera entre desarrollo y evolución tiende a difuminarse).
- No hay un proceso "mejor": depende de tipo de software, tamaño, ámbito, equipo.
- Coste de corregir un error: crece cuanto más tarde se detecta.
- Diseño arquitectónico (qué módulos) vs. diseño detallado (comportamiento de cada módulo).
- Mantenimiento: correctivo / perfectivo / adaptativo.

2.2 · Modelos secuenciales

- Cascada: se completa una fase antes de pasar a la siguiente; documentación exhaustiva aprobada como entrada de la fase siguiente.
- V: variante de cascada centrada en verificación/validación; cada fase de desarrollo tiene su fase de prueba asociada.
- Ambos: poca adaptación al cambio, feedback tardío, alta trazabilidad, útiles con requisitos estables o entornos regulados (por ejemplo, sector aeronáutico, salud, automoción...).

2.3 · Modelos iterativos e incrementales

- Iteración = ciclo de trabajo (análisis-diseño-implementación-pruebas). Incremento = versión funcional resultante.
- Desarrollo evolutivo: exploratorio (el prototipo se convierte en producto) vs. desechable (solo para entender requisitos).
- Espiral (Boehm, 1988): 4 sectores por ciclo (objetivos y restricciones · evaluación y reducción de riesgos · desarrollo y validación · planificación del siguiente ciclo), foco en gestión de riesgos; complejo, no apto para proyectos pequeños. Muy poco utilizado en la industria.
- UP (Proceso Unificado, Jacobson, Booch y Rumbaugh, 1999): tres pilares — dirigido por casos de uso + centrado en arquitectura + iterativo e incremental.
- UP organiza el ciclo en 4 fases (Inicio, Elaboración, Construcción, Transición); las iteraciones ocurren dentro de las fases, sobre todo en Elaboración y Construcción. Fase ≠ iteración.

2.4 · Modelos ágiles

- Origen: rigidez de los modelos tradicionales + naturaleza creativa (no solo técnica) del diseño de software.

- Manifiesto Ágil (2001): 4 valores, 12 principios. No implica ausencia de proceso: exige estructura y disciplina.
- Scrum: 5 valores (compromiso, foco, apertura, respeto, valentía); roles (Product Owner, Scrum Master, Developers); artefactos (Product Backlog, Sprint Backlog, Incremento); eventos (Sprint, Planning, Daily, Review, Retrospective). El Incremento debe cumplir la Definition of Done (DoD). No prescribe prácticas técnicas.
- XP: 5 valores (simplicidad, comunicación, feedback, coraje, respeto); prácticas técnicas (TDD, refactorización, pair programming, propiedad colectiva del código, integración continua); cliente in situ como rasgo distintivo frente a Scrum. Sin roles definidos estrictamente.
- Historia de usuario: formato + criterios de aceptación (se profundizará en el Tema 3).

2.5 · Gestión de configuración y control de versiones

- Objetivo: trabajar sobre una base coherente y trazable cuando varias personas modifican el sistema.
- Conceptos: repositorio, commit, branch, merge, tag/release.
- Buenas prácticas: commits pequeños y frecuentes, mensajes claros, ramas por funcionalidad, revisión antes de fusionar, etiquetar versiones estables.

2.6 · CI/CD

- CI: cada commit dispara compilación + pruebas automáticas. No se despliega nada, solo se valida.
- Entrega continua: además, se deja lista una versión desplegable; el despliegue sigue siendo manual.
- Despliegue continuo: el despliegue a producción también es automático.
- Herramientas: GitHub Actions, GitLab CI/CD, Jenkins (y otras: Travis CI, CircleCI, Bitbucket Pipelines, Azure Pipelines).
- CI/CD es la palanca operativa de DevOps; DevOps es la filosofía/cultura, CI/CD la ejecuta.

2.7 · IA en el proceso

- Dos usos distintos: IA como parte del producto (se diseña y evalúa) vs. IA como herramienta del proceso (asiste al ingeniero). El tema trata el segundo.
- Interviene en requisitos, análisis/diseño, implementación, pruebas, documentación, mantenimiento y gestión — no solo en programar.
- No es un modelo de proceso nuevo: se integra en los existentes (apoyo tipo pair programming en ágil; generación/revisión de pruebas en CI/CD).

- Riesgos: alucinaciones, seguridad, propiedad intelectual, confidencialidad, sesgos, dependencia.
- La responsabilidad final del resultado entregado es siempre de la persona, nunca de la herramienta. En entornos regulados, el uso de IA debe documentarse.

Tabla de decisión: ¿qué modelo de proceso encaja?

Característica del proyecto	Modelo(s)	Por qué
Requisitos estables, bien conocidos desde el inicio	Cascada	Permite planificar y documentar fase a fase sin esperar cambios
Entorno normativo estricto o certificación (médico, aeronáutico, banca, administración)	V	Trazabilidad directa requisito ↔ prueba, verificación explícita en cada nivel
Proyecto complejo con requisitos identificables por casos de uso y necesidad de una arquitectura sólida consolidada progresivamente	UP	Combina trazabilidad (casos de uso), iteración controlada y consolidación evolutiva de la arquitectura
Requisitos cambiantes o poco claros al inicio	Ágil (Scrum/XP)	Los requisitos se capturan y ajustan de forma evolutiva, con entregas frecuentes
Equipo pequeño (2–10), colaboración cercana con cliente, énfasis en calidad técnica del código	XP	Prácticas técnicas explícitas (TDD, pair programming, integración continua)
Necesidad de organización del trabajo en ciclos con roles claros, tamaño de equipo algo mayor	Scrum	Roles y artefactos definidos; no impone prácticas técnicas concretas
Sistema embebido o de tiempo real con entorno estable y bien conocido	Cascada	El entorno físico y las restricciones de tiempo real quedan fijadas desde el diseño; los cambios tardíos son especialmente costosos por la integración hardware-software, lo que favorece especificar y validar por fases antes de comprometer el desarrollo
Sistema de gestión empresarial con procesos normalizados (inventario, nóminas...)	Cascada o Ágil	Cascada si el alcance está realmente cerrado y estable; en la práctica actual es frecuente optar por ágil, porque las reglas concretas evolucionan (normativa, tarifas, integraciones) y permite entregar valor desde las primeras iteraciones

Nota: esta tabla es una simplificación útil a efectos pedagógicos. La elección de un modelo de proceso para un proyecto concreto debe tener en cuenta múltiples aspectos. En la realidad, un mismo proyecto suele presentar varias de estas características a la vez, a menudo combinadas de forma poco evidente, por lo que la decisión rara vez se reduce a aplicar una sola fila de la tabla de forma aislada.

Confusiones habituales

- **Iteración vs. incremento** — la iteración es el ciclo de trabajo (proceso); el incremento es el resultado tangible de ese ciclo (producto parcial funcional).
- **Verificación vs. validación** — validación: ¿es el sistema que el cliente necesita?
Verificación: ¿se ha construido conforme a la especificación? Un sistema puede pasar la validación funcional y aun así fallar en verificación (fiabilidad, escalabilidad...).
- **"Ágil" = sin proceso** — los métodos ágiles no son improvisación; exigen estructura, disciplina y reflexión continua (retrospectivas). Delegan decisiones en el equipo, no las eliminan.
- **Entrega continua vs. despliegue continuo** — en entrega continua el sistema queda listo para desplegar pero el despliegue es manual; en despliegue continuo el propio despliegue a producción es automático.
- **Fase vs. iteración (UP)** — la fase es una etapa del ciclo de vida (Inicio/Elaboración/Construcción/Transición); la iteración es un ciclo de trabajo que ocurre dentro de una fase.
- **UML ≠ proceso de desarrollo** — UML es un lenguaje de modelado; no especifica cómo organizar el desarrollo. El UP es el modelo de proceso que se construyó después, por los mismos autores, para responder a esa pregunta.
- **Big bang no es integración continua** — ambas implican "juntar componentes", pero big bang los ensambla todos al final (desaconsejado); integración continua los incorpora automáticamente en cuanto se producen, con pruebas frecuentes.

Glosario mínimo

Proceso de desarrollo: conjunto organizado de actividades que transforma una necesidad en un sistema software.

Iteración: ciclo de trabajo completo (análisis, diseño, implementación, pruebas).

Incremento: versión funcional del sistema resultante de una iteración.

Verificación: comprobación de que el sistema se ajusta a la especificación.

Validación: comprobación de que el sistema construido es el que necesita el cliente.

Sprint: ciclo de trabajo de duración fija en Scrum (2–4 semanas).

Product / Sprint Backlog: lista priorizada de todo lo que necesita el producto / subconjunto que el equipo aborda en un Sprint.

Definition of Done (DoD): criterios acordados que debe cumplir un incremento para considerarse completo.

TDD: escribir la prueba automatizada antes que el código que la hace pasar.

Refactorización: mejorar la estructura interna del código sin alterar su comportamiento externo.

Programación en parejas: dos personas, una programa y otra revisa, con roles alternos.

Propiedad colectiva del código: cualquier miembro del equipo puede modificar cualquier parte del sistema.

CI (Integración Continua): cada commit se compila y prueba automáticamente.

Entrega continua: cada cambio validado queda listo para desplegar; el despliegue es manual.

Despliegue continuo: el despliegue a producción también es automático.

Pipeline: secuencia automatizada de pasos (compilar, probar, desplegar) que se ejecuta ante cada cambio.

Repositorio / commit / branch / merge / tag: espacio de código versionado / registro de cambios / línea de desarrollo paralela / fusión de ramas / marca sobre un commit como versión relevante.

DevOps: modelo organizativo y cultural que integra desarrollo y operaciones para entregar valor de forma continua.

Sobre el uso de Inteligencia Artificial en la elaboración de este material

En la elaboración de esta hoja de esenciales y del mapa conceptual que la acompaña se ha utilizado un asistente de Inteligencia Artificial generativa (Claude, Anthropic), en el nivel 3 ("AI Collaboration") de la Escala AIAS (Perkins, Furze, Roe y MacVaugh, 2024), con el siguiente alcance:

- Síntesis, reestructuración y condensación del contenido del Tema 2 —ya redactado y revisado previamente por el profesor— en un formato de repaso: objetivos de aprendizaje, esenciales por bloque, tabla de decisión, confusiones habituales, glosario y mapa conceptual.
- El profesor ha revisado, corregido y modificado directamente el resultado generado por la IA en varias rondas sucesivas (ampliaciones de contenido, ajustes de redacción, eliminación y reformulación de filas de la tabla de decisión).
- En este resumen no se ha incorporado contenido ajeno a los apuntes fuente ni otro conocimiento externo.

Cualquier duda sobre el alcance de este uso puede consultarse directamente con el profesor.

Mapa conceptual

